ChainEdgeLLM — Multi-dimensional Threat Activation Dataset Generator & Classifier

多维威胁激活状态数据集生成器与分类器

> **English** | 中文

A complete pipeline for generating labeled hidden-state datasets from LLM decoder layers and training threat detection classifiers for decentralized edge computing security.

Project Overview

This project consists of two main modules:

1. **threat_dataset** — Generates labeled datasets by capturing hidden states from LLM decoder layers and simulating 5 types of threats/attacks
2. **classify** — Trains and evaluates classical ML and neural network classifiers to detect threats from hidden-state vectors

Complete Workflow

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────┐
│                                STEP 1: Data Generation                                           │
│                              (threat_dataset module)                                             │
├──────────────────────────────────────────────────────────────────────────────────────────────────┤
│                                                                                                  │
│    ┌──────────────┐     ┌──────────────┐     ┌──────────────┐     ┌──────────────┐             │
│    │ Load Model   │──→│  Register    │──→│   Process    │──→│  Generate    │             │
│    │  + Dataset   │     │    Hooks     │     │ Instructions│     │  Threats     │             │
│    └──────────────┘     └──────────────┘     └──────────────┘     └──────────────┘             │
│                                                                          │                       │
│                                                                          ↓                       │
│   Output: Parquet files (hidden_state_vector, label, layer_index...)                             │
│                                                                                                  │
│                                                                                                  │
```

```
└──────────────
```

```
  ├── threats.py           # 5 threat-label generators (pure NumPy)
  ├── buffer_io.py         # RecordBuffer with periodic Parquet flush
(pyarrow)
  └── main.py              # CLI orchestration loop
```

CLI Usage

```
python -m threat_dataset [OPTIONS]
```

All CLI Options

Flag	Type	Choices / Default	Description
`--model`	str	`meta-llama/Meta-Llama-3-8B`	HuggingFace/ModelScope model identifier
`--dataset-size`	int	`5000`	Number of Alpaca instructions to process
`--output-dir`	str	`output`	Directory for Parquet part files
`--dtype`	str	`bf16`	Model precision: `bf16` or `fp16`
`--buffer-size`	int	`10000`	Records to buffer before flushing to disk
`--device`	str	`cuda`	PyTorch device (`cuda`, `cpu`, etc.)
`--log-dir`	str	`logs`	Directory for log files
`--source`	str	`huggingface`	Model source: `huggingface` or `modelscope`
`--step`	str	`all`	Run step: `download` (cache model only), `inference` (generate dataset), or `all`
`--dataset-name`	str	See below	Dataset identifier (default: `tatsu-lab/alpaca` for HF, `AI-ModelScope/alpaca-gpt4-data-en` for MS)

Parameter Details

`--model` — Model identifier from HuggingFace Hub or ModelScope

- HuggingFace examples: `meta-llama/Meta-Llama-3-8B`, `Qwen/Qwen1.5-7B`

- ModelScope examples: `LLM-Research/Meta-Llama-3-8B`

`--source` — Where to download the model from

- `huggingface`: Use HuggingFace `transformers` library (default)
- `modelscope`: Use ModelScope `msdatasets` library (for users in China)

`--step` — Control execution flow

- `download`: Only download and cache the model (no dataset generation)
- `inference`: Generate dataset using already-cached model
- `all`: Download model then generate dataset (default)

`--dtype` — Model precision for inference

- `bf16`: BFloat16 (default, recommended for Ampere GPUs)
- `fp16`: Float16 (alternative for older GPUs)

`--dataset-name` — Override the default dataset

- HuggingFace default: `tatsu-lab/alpaca`
- ModelScope default: `AI-ModelScope/alpaca-gpt4-data-en`
- Custom: Any Alpaca-format dataset with `instruction` and `input` fields

Quick Start Examples

```
# 1. Small test (5 instructions) - HuggingFace source
python -m threat_dataset \
    --model meta-llama/Meta-Llama-3-8B \
    --dataset-size 5 \
    --step all

# 2. Small test - ModelScope source (China)
python -m threat_dataset \
    --model LLM-Research/Meta-Llama-3-8B \
    --dataset-size 5 \
    --source modelscope \
    --step all

# 3. Only download model (prepare for later inference)
python -m threat_dataset \
    --model meta-llama/Meta-Llama-3-8B \
    --step download

# 4. Generate dataset using cached model (resume-capable)
python -m threat_dataset \
    --model meta-llama/Meta-Llama-3-8B \
    --dataset-size 10000 \
    --step inference \
    --source modelscope \
    --output-dir /path/to/data
```

```
# 5. Full production run with Qwen model
python -m threat_dataset \
    --model Qwen/Qwen1.5-7B \
    --dataset-size 10000 \
    --dtype bf16 \
    --buffer-size 10000 \
    --device cuda \
    --output-dir ./output
```

Output Schema (Parquet)

Column	Type	Description
`sample_id`	String	Unique ID, e.g. `00001_L015_T2`
`model_name`	String	Short model name, e.g. `Meta-Llama-3-8B`
`instruction_id`	Int32	Index of the source instruction
`layer_index`	Int32	Decoder layer index (0-based)
`label`	Int32	Threat label (0–4)
`label_name`	String	Human-readable label name
`hidden_state_vector`	List	Feature vector of length `hidden_size` (4096 for Llama3-8B)

Label Definitions

Label	Name	Description
0	`honest`	Genuine decoder output
1	`silent_precision_downgrade`	Simulated INT4 quantization round-trip
2	`identity_forgery`	Layer input passed through as output (skip attack)
3	`random_noise`	Gaussian noise matching output statistics (Byzantine fault)
4	`adversarial_perturbation`	Honest output + 5% std Gaussian noise

Resume Capability

The tool supports resuming interrupted runs via `resume_state.json` in the output directory:

```
# If interrupted, simply re-run the same command
# It will automatically continue from the last completed instruction
python -m threat_dataset \
    --model LLM-Research/Meta-Llama-3-8B \
    --dataset-size 10000 \
    --step inference \
```

```
    --source modelscope \
    --output-dir /root/autodl-tmp/data
```

Module 2: classify (Threat Classification)

Train and evaluate ML classifiers to detect threats from hidden-state vectors.

Directory Structure

```
classify/
├── __init__.py          # Package metadata
├── __main__.py          # python -m classify entry
├── main.py              # CLI and classification pipeline
├── nn_models.py         # PyTorch MLP classifiers (sklearn-compatible)
└── logging_setup.py     # Logging configuration
```

CLI Usage

```
python -m classify [OPTIONS]
```

All CLI Options

Flag	Type	Choices / Default	Description
`--data-dir`	str	`output`	Directory containing .parquet data files
`--task`	str	`cross-layer`	Classification mode: `per-layer` or `cross-layer`
`--pca-dim`	int	`128`	PCA components for dimensionality reduction (0 to disable)
`--val-size`	float	`0.15`	Validation set ratio (0.0–1.0)
`--test-size`	float	`0.15`	Test set ratio (0.0–1.0)
`--seed`	int	`42`	Random seed for reproducibility

Parameter Details

`--task` — Classification strategy

- `cross-layer`: Train a **single classifier** on hidden states from all layers mixed together (layer origin is unknown). Tests generalization across layers.

- `per-layer`: Train **one classifier per decoder layer**. Each classifier only sees data from its assigned layer. Tests layer-specific threat signatures.

`--pca-dim` — Dimensionality reduction

- Default: `128` (reduce 4096-dim vectors to 128-dim)
- Set to `0` to disable PCA (use full dimension)
- Automatically skipped if `pca_dim >= original_dim`

`--val-size` / **`--test-size`** — Data split ratios

- Train set = `1 - val_size - test_size`
- Uses stratified splitting to maintain class balance

Quick Start Examples

```
# 1. Cross-layer classification (default) with PCA
python -m classify \
    --data-dir output \
    --task cross-layer \
    --pca-dim 128

# 2. Per-layer classification (train 32 separate models for 32 layers)
python -m classify \
    --data-dir output \
    --task per-layer \
    --pca-dim 128

# 3. No PCA (use full 4096-dim vectors)
python -m classify \
    --data-dir output \
    --pca-dim 0

# 4. Custom split ratios
python -m classify \
    --data-dir output \
    --val-size 0.1 \
    --test-size 0.2 \
    --seed 123
```

Classifiers Used

Classifier	Library	Notes
LogisticRegression	sklearn	With StandardScaler, multinomial
RandomForest	sklearn	200 estimators
SVM_RBF	sklearn	RBF kernel, One-vs-Rest
KNN	sklearn	k=5 neighbors

Classifier	Library	Notes
LightGBM	lightgbm	If installed, 200 estimators
MLP_Small	PyTorch	[128] hidden units
MLP_Medium	PyTorch	[256, 128] hidden units
MLP_Large	PyTorch	[512, 256, 128] hidden units

Output Files

Results are saved to `{data-dir}/classify_results/`:

```
output/classify_results/
├── summary_cross_layer.csv          # Metrics summary (cross-layer task)
├── summary_per_layer.csv            # Metrics summary (per-layer task)
├── cm_cross_layer_{classifier}_val.png      # Confusion matrices
├── cm_cross_layer_{classifier}_test.png
├── report_cross_layer_{classifier}_test.txt # Classification reports
└── layer_{N}/                       # Per-layer results (per-layer task)
    ├── cm_layer_{N}_{classifier}_val.png
    ├── cm_layer_{N}_{classifier}_test.png
    └── report_layer_{N}_{classifier}_test.txt
```

Complete Pipeline Example

```
# Step 1: Generate dataset (ModelScope source, 10k instructions)
python -m threat_dataset \
    --model LLM-Research/Meta-Llama-3-8B \
    --dataset-size 10000 \
    --step all \
    --source modelscope \
    --output-dir ./data \
    --dtype bf16

# Step 2a: Cross-layer classification
python -m classify \
    --data-dir ./data \
    --task cross-layer \
    --pca-dim 128

# Step 2b: Per-layer classification
python -m classify \
    --data-dir ./data \
    --task per-layer \
    --pca-dim 128

# Check results
```

```
cat ./data/classify_results/summary_cross_layer.csv
ls ./data/classify_results/*.png  # Confusion matrices
```

Troubleshooting

OOM (Out of Memory)

- The tool extracts only the **last token's** hidden state per layer
- Try `--dtype fp16` or use a smaller model
- Reduce `--buffer-size` if CPU RAM is tight

Type / Compatibility Errors

- Requires `torch>=2.1` for proper bfloat16 support
- Check inspection pass logs for `input[0]_shape` and `output[0]_shape`
- Add custom layer paths to `LAYER_ATTR_PATHS` in `config.py` if needed

Slow Throughput

- Batch size is fixed at 1 (no padding) for correctness
- Expected: ~2–4 instructions/second on A100 with 7B model

中文文档

> English | **中文**

用于在去中心化边缘计算场景中生成大型语言模型解码层威胁激活数据集，并训练威胁检测分类器的完整流水线。

项目概述

本项目包含两个核心模块：

1. **threat_dataset** — 通过挂载 LLM 各 Decoder 层的 Forward Hook，截获隐藏状态并生成 5 种威胁/攻击类型的标记数据集
2. **classify** — 训练并评估经典机器学习和神经网络分类器，从隐藏状态向量中检测威胁

完整工作流程

```
┌──────────────
```

```
│                                                                                │
└──┘
│
│
│    ┌──────────────┐     ┌──────────────┐     ┌──────────────┐     ┌──────────────┐
│
│    │  加载模型    │──→│   注册Hook   │──→│   处理指令   │──→│   生成威胁   │  │
│    │  和数据集    │     │              │     │              │     │              │
│
│    └──────────────┘     └──────────────┘     └──────────────┘     └──────────────┘
│
│                                                                        │
│
│                                                                        ↓
│
│    输出：Parquet 文件 (hidden_state_vector, label, layer_index...)
│
│
│
└──────────────────────────────────────────────────────────────────────────────┘
└──┘
                                        │
                                        ↓
┌──────────────────────────────────────────────────────────────────────────────┐
└──┐
│                        第二步：威胁分类
│
│                           (classify 模块)
│
├──────────────────────────────────────────────────────────────────────────────┤
└──┤
│
│
│    ┌──────────────┐     ┌──────────────┐     ┌──────────────┐     ┌──────────────┐
│
│    │   加载数据   │──→│  PCA (可选) │──→│  训练/测试   │──→│   评估       │  │
│    │   Parquet    │     │   降维       │     │    划分      │     │  分类器      │
│
│    └──────────────┘     └──────────────┘     └──────────────┘     └──────────────┘
│
│                                                                        │
│
│                                                                        ↓
│
│    输出：指标 CSV、混淆矩阵、分类报告                                          │
│
│
└──────────────────────────────────────────────────────────────────────────────┘
└──┘
```

模块一：threat_dataset (数据生成)

从 LLM 解码层生成多维威胁激活状态数据集。

目录结构

```
threat_dataset/
├── __init__.py          # 包元数据
├── __main__.py          # python -m threat_dataset 入口
├── config.py            # 常量、标签映射、Parquet Schema、CLI 参数、RunConfig
├── logging_setup.py     # 双输出日志（控制台 INFO + 文件 DEBUG）+ 内存监控
├── model_loader.py      # HF 模型/Tokenizer 加载、Decoder 层发现、结构探查
├── hooks.py             # Forward Hook 工厂、LayerStateCapture、最后 Token
提取
├── threats.py           # 5 种威胁标签生成函数（纯 NumPy）
├── buffer_io.py         # RecordBuffer，定期分片写入 Parquet（pyarrow）
└── main.py              # CLI 主循环编排
```

CLI 使用方法

```
python -m threat_dataset [选项]
```

所有 CLI 参数

参数	类型	可选值 / 默认值	说明
`--model`	字符串	`meta-llama/Meta-Llama-3-8B`	HuggingFace/ModelScope 模型标识符
`--dataset-size`	整数	`5000`	要处理的 Alpaca 指令数量
`--output-dir`	字符串	`output`	Parquet 分片文件输出目录
`--dtype`	字符串	`bf16`	模型精度：`bf16` 或 `fp16`
`--buffer-size`	整数	`10000`	落盘前最大缓冲记录数
`--device`	字符串	`cuda`	PyTorch 运行设备（`cuda`、`cpu` 等）

参数	类型	可选值 / 默认值	说明
`--log-dir`	字符串	`logs`	日志文件目录
`--source`	字符串	`huggingface`	模型来源：`huggingface` 或 `modelscope`
`--step`	字符串	`all`	执行步骤：`download`（仅缓存模型）、`inference`（生成数据集）、`all`
`--dataset-name`	字符串	见下文	数据集标识符

参数详细说明

`--model` — 模型标识符

- HuggingFace 示例：`meta-llama/Meta-Llama-3-8B`、`Qwen/Qwen1.5-7B`
- ModelScope 示例：`LLM-Research/Meta-Llama-3-8B`

`--source` — 模型下载来源

- `huggingface`：使用 HuggingFace `transformers` 库（默认）
- `modelscope`：使用 ModelScope `msdatasets` 库（国内用户推荐）

`--step` — 控制执行流程

- `download`：仅下载并缓存模型（不生成数据集）
- `inference`：使用已缓存的模型生成数据集
- `all`：下载模型然后生成数据集（默认）

`--dtype` — 推理精度

- `bf16`：BFloat16（默认，推荐用于 Ampere GPU）
- `fp16`：Float16（旧 GPU 备选）

`--dataset-name` — 覆盖默认数据集

- HuggingFace 默认：`tatsu-lab/alpaca`
- ModelScope 默认：`AI-ModelScope/alpaca-gpt4-data-en`
- 自定义：任何包含 `instruction` 和 `input` 字段的 Alpaca 格式数据集

快速开始示例

```
# 1. 小规模测试（5 条指令）- HuggingFace 源
python -m threat_dataset \
    --model meta-llama/Meta-Llama-3-8B \
    --dataset-size 5 \
    --step all

# 2. 小规模测试 - ModelScope 源（国内）
python -m threat_dataset \
    --model LLM-Research/Meta-Llama-3-8B \
    --dataset-size 5 \
    --source modelscope \
    --step all

# 3. 仅下载模型（为后续推理做准备）
python -m threat_dataset \
    --model meta-llama/Meta-Llama-3-8B \
    --step download

# 4. 使用缓存的模型生成数据集（支持断点续传）
python -m threat_dataset \
    --model LLM-Research/Meta-Llama-3-8B \
    --dataset-size 10000 \
    --step inference \
    --source modelscope \
    --output-dir /path/to/data

# 5. 使用 Qwen 模型的完整生产运行
python -m threat_dataset \
    --model Qwen/Qwen1.5-7B \
    --dataset-size 10000 \
    --dtype bf16 \
    --buffer-size 10000 \
    --device cuda \
    --output-dir ./output
```

输出数据结构（Parquet Schema）

列名	类型	说明
`sample_id`	字符串	唯一标识符，如 `00001_L015_T2`
`model_name`	字符串	模型短名称，如 `Meta-Llama-3-8B`
`instruction_id`	整数	原始指令在数据集中的索引
`layer_index`	整数	解码器层索引（从 0 开始）
`label`	整数	威胁分类标签（0–4）
`label_name`	字符串	标签文本名称
`hidden_state_vector`	List	特征向量，长度为 `hidden_size`（Llama3-8B 为 4096）

标签定义

标签	名称	描述
0	`honest`	诚实执行，真实输出
1	`silent_precision_downgrade`	模拟 INT4 对称量化后反量化的截断误差
2	`identity_forgery`	跳层逃逸，直接返回上层输入
3	`random_noise`	与输出同均值/方差的随机高斯噪声（拜占庭故障）
4	`adversarial_perturbation`	诚实输出 + 5% 标准差高斯扰动（微小投毒）

断点续传功能

工具通过输出目录中的 `resume_state.json` 支持中断后恢复：

```
# 如果中断了，只需重新运行相同命令
# 它会自动从最后完成的指令继续
python -m threat_dataset \
    --model LLM-Research/Meta-Llama-3-8B \
    --dataset-size 10000 \
    --step inference \
    --source modelscope \
    --output-dir /root/autodl-tmp/data
```

模块二：classify (威胁分类)

从隐藏状态向量训练并评估机器学习分类器以检测威胁。

目录结构

```
classify/
├── __init__.py          # 包元数据
├── __main__.py          # python -m classify 入口
├── main.py              # CLI 和分类流水线
├── nn_models.py         # PyTorch MLP 分类器（兼容 sklearn 接口）
└── logging_setup.py     # 日志配置
```

CLI 使用方法

```
python -m classify [选项]
```

所有 CLI 参数

参数	类型	可选值 / 默认值	说明
`--data-dir`	字符串	`output`	包含 .parquet 数据文件的目录
`--task`	字符串	`cross-layer`	分类模式：`per-layer` 或 `cross-layer`
`--pca-dim`	整数	`128`	PCA 降维后的维度（0 表示禁用）
`--val-size`	浮点数	`0.15`	验证集比例（0.0–1.0）
`--test-size`	浮点数	`0.15`	测试集比例（0.0–1.0）
`--seed`	整数	`42`	随机种子，用于结果可复现

参数详细说明

`--task` — 分类策略

- `cross-layer`：在混合了所有层的隐藏状态上训练**单一分类器**（层来源未知）。测试跨层的泛化能力。
- `per-layer`：为每个解码层训练**独立分类器**。每个分类器只看到分配给它的层的数据。测试层特定的威胁特征。

`--pca-dim` — 降维

- 默认：`128`（将 4096 维向量降至 128 维）
- 设置为 `0` 禁用 PCA（使用完整维度）
- 如果 `pca_dim >= 原始维度`，自动跳过

`--val-size` / `--test-size` — 数据划分比例

- 训练集 = `1 - val_size - test_size`
- 使用分层划分以保持类别平衡

快速开始示例

```
# 1. 跨层分类（默认）带 PCA
python -m classify \
    --data-dir output \
    --task cross-layer \
    --pca-dim 128

# 2. 逐层分类（为 32 层训练 32 个独立模型）
python -m classify \
    --data-dir output \
    --task per-layer \
    --pca-dim 128

# 3. 不使用 PCA（使用完整 4096 维向量）
python -m classify \
    --data-dir output \
    --pca-dim 0

# 4. 自定义划分比例
```

```
python -m classify \
    --data-dir output \
    --val-size 0.1 \
    --test-size 0.2 \
    --seed 123
```

使用的分类器

分类器	库	说明
LogisticRegression	sklearn	带 StandardScaler，多分类
RandomForest	sklearn	200 棵决策树
SVM_RBF	sklearn	RBF 核，一对多策略
KNN	sklearn	k=5 邻居
LightGBM	lightgbm	如已安装，200 棵决策树
MLP_Small	PyTorch	[128] 隐藏单元
MLP_Medium	PyTorch	[256, 128] 隐藏单元
MLP_Large	PyTorch	[512, 256, 128] 隐藏单元

输出文件

结果保存到 `{data-dir}/classify_results/`：

```
output/classify_results/
├── summary_cross_layer.csv          # 指标汇总（跨层任务）
├── summary_per_layer.csv            # 指标汇总（逐层任务）
├── cm_cross_layer_{classifier}_val.png      # 混淆矩阵
├── cm_cross_layer_{classifier}_test.png
├── report_cross_layer_{classifier}_test.txt # 分类报告
└── layer_{N}/                       # 逐层结果（逐层任务）
    ├── cm_layer_{N}_{classifier}_val.png
    ├── cm_layer_{N}_{classifier}_test.png
    └── report_layer_{N}_{classifier}_test.txt
```

完整流水线示例

```
# 第一步：生成数据集（ModelScope 源，10k 条指令）
python -m threat_dataset \
    --model LLM-Research/Meta-Llama-3-8B \
    --dataset-size 10000 \
    --step all \
    --source modelscope \
```

```
    --output-dir ./data \
    --dtype bf16

# 第二步 a：跨层分类
python -m classify \
    --data-dir ./data \
    --task cross-layer \
    --pca-dim 128

# 第二步 b：逐层分类
python -m classify \
    --data-dir ./data \
    --task per-layer \
    --pca-dim 128

# 查看结果
cat ./data/classify_results/summary_cross_layer.csv
ls ./data/classify_results/*.png  # 混淆矩阵
```

常见问题排查

显存不足（OOM）

- 工具仅在 Hook 内部提取**最后一个词元**的隐状态
- 尝试 `--dtype fp16` 或换用参数量更小的模型
- CPU 内存紧张时，调低 `--buffer-size`

类型兼容问题

- 需要 `torch>=2.1` 以支持 bfloat16
- 检查结构探查日志中的 `input[0]_shape` 和 `output[0]_shape`
- 如需支持非标准架构，请在 `config.py` 的 `LAYER_ATTR_PATHS` 中添加正确的层路径

推理速度慢

- Batch Size 强制为 1（无填充）以确保正确性
- A100 上 7B 模型处理 5000 条指令，预计约 2–4 条/秒

License

本项目代码遵循相应开源许可证。原始模型（Meta-Llama-3-8B 等）受其自身许可证约束。

Acknowledgments

- 基础模型：Meta-Llama-3-8B by Meta AI
- 指令数据集：Alpaca GPT-4 基于 Stanford Alpaca